

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192324237
Name	D.Manjunadha

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I _D.Manjunadha(19232423) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:Manjunadha

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
		with some justification			
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

Q1)

Scenario: AI-Powered Drone for Emergency Medicine Delivery in a Flood-Affected Region

1. Behavior of Greedy Best-First Search (GBFS) in this Scenario

Greedy Best-First Search selects the next node based only on the heuristic value $h(n)$, which estimates the remaining distance to the goal. It always expands the node that appears closest to the destination without considering the travel cost already incurred.

How GBFS Works

- Uses only the heuristic function:
 $f(n) = h(n)$
- Chooses the path with the smallest estimated distance to the goal.
- Ignores accumulated travel cost such as fuel usage, weather delays, and difficult terrain.

Behavior in the Flood Scenario

The drone will continuously move toward the destination using the route that appears geographically closest. If a straight road leads directly to the hospital but becomes flooded, the drone may still attempt to move toward that blocked route because the heuristic indicates it is the shortest path. Only after discovering the blockage will it search for another route.

2. How A* Search Handles the Same Situation

A* Search combines the actual travel cost from the starting point with the estimated remaining distance to the destination. It evaluates each possible path more intelligently.

Evaluation Function

$$f(n) = g(n) + h(n)$$

Behavior in the Flood Scenario

The drone considers:

- Distance already traveled.
- Weather-related travel costs.
- Terrain difficulty.
- Estimated remaining distance.

If a road becomes flooded, the movement cost increases significantly or becomes infinite. A* recalculates the best available path by considering both current travel cost and future estimated cost.

Strengths

- Produces optimal routes when the heuristic is admissible.
- Considers both travel distance and environmental costs.
- Avoids unnecessarily expensive routes.

3. Impact of Heuristic Accuracy on Both Algorithms

The quality of the heuristic significantly affects the performance of both search algorithms.

For Greedy Best-First Search

when the heuristic is highly accurate:

- quickly reaches the destination.
- expands fewer nodes.
- performs efficiently.

For A* Search

when the heuristic is accurate:

- finds the optimal route efficiently.
- explores fewer unnecessary nodes.
- reduces computation time.

4. . Effect of Dynamic Changes (Blocked Paths)

Flood conditions constantly change, making the search environment dynamic.

Effect on Greedy Best-First Search

- Continues moving toward blocked regions because they appear closer.
- Detects obstacles only after reaching them.
- Requires repeated backtracking.
- Delivery time increases.
- Higher battery consumption.

Effect on A* Search

- Updates travel costs when obstacles are detected.
- Recalculates a new optimal route.
- Better adapts to changing conditions.
- Minimizes unnecessary travel.
- Improves safety and reliability.

5. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

A Search is the most suitable algorithm.

Justification

1. Real-Time Constraints

Although Greedy Best-First Search makes decisions faster, repeated wrong choices and backtracking often increase total delivery time. A* requires slightly more computation but usually reaches the destination faster overall because it avoids poor routes.

2. Optimality

A* guarantees the optimal path when the heuristic is admissible. Greedy Best-First Search does not guarantee the shortest or least expensive path.

3. Safety

A* considers weather conditions, terrain difficulty, and blocked paths while planning routes, reducing the risk of drone failure or delays. Greedy Search may direct the drone into hazardous areas.

2Q)

Scenario: AI-Based Smart Traffic Signal Optimization in a Smart City:

- I. Explain how Hill Climbing would work in this scenario and identify its limitations

Working

The AI begins with an initial set of traffic signal timings.

It then:

1. Measures total traffic delay.
2. Slightly modifies one or more signal timings.
3. Evaluates whether traffic performance improves.

4. Accepts the new solution only if it is better.
5. Repeats the process until no further improvement is possible.

Advantages

- Simple to implement.
- Fast convergence.
- Low memory requirement.
- Suitable for small optimization problems.
- Produces quick improvements.

Limitations

- Easily gets trapped in local optimum solutions.
- Cannot guarantee the global optimum.
- Stops when no neighboring solution is better.

II. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

1. Local Maxima

A local maximum is a solution that is better than all nearby solutions but is not the best overall solution.

Real-World Interpretation

One intersection may have perfectly optimized signal timing.

However, neighboring intersections remain congested.

Hill Climbing stops because no small adjustment improves the local intersection, even though a much better city-wide solution exists.

2. Plateaus

A plateau is a flat region where many neighboring solutions have identical evaluation values.

Real-World Interpretation

Changing green time from:

- 40 seconds
- 41 seconds
- 42 seconds

may produce exactly the same traffic delay.

III. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing:

Working

- Starts with a high temperature.
- Explores many possible solutions.
- Sometimes accepts worse solutions to escape local optima.
- Temperature gradually decreases.

Advantages

- Escapes local maxima.
- Explores a larger search space.
- Better chance of finding the global optimum.
- Effective for dynamic optimization.

Genetic Algorithm (GA)

Working

1. Generate multiple random traffic signal plans.

2. Evaluate each plan using the objective function.
3. Select the best-performing plans.
4. Perform crossover to combine good features.

Advantages

- Searches many solutions simultaneously.
- Avoids local optima.
- Works well for very large search spaces.

IV. Recommend the best approach and justify your choice based on scalability and adaptability

Genetic Algorithms (GA) are the most suitable approach.

1. Scalability.

Genetic Algorithms efficiently explore very large search spaces, whereas Hill Climbing becomes trapped in local optima.

2. Adaptability

Traffic conditions change continuously due to accidents, weather, and rush hours.

Genetic Algorithms can continuously evolve better signal timing plans as traffic data changes.

3. Global Optimization

GA explores many candidate solutions simultaneously.

This significantly increases the probability of finding a near-global optimum compared to Hill Climbing.

3Q)

Scenario: Autonomous Mars Rover – Online Search Agent

I. Explain why this problem requires an online search agent instead of offline search

An **offline search agent** assumes that the entire environment is known before planning begins. It has complete information about all possible paths, obstacles, and the goal location. Using this information, it computes the optimal path first and then executes it. Such an approach is effective only in static and fully observable environments.

In contrast, the Mars rover operates in an **unknown and unpredictable environment**. Before moving, it does not know the exact positions of craters, rocks, steep slopes, or other obstacles. Its sensors reveal only the nearby surroundings as it explores. Therefore, it cannot generate a complete path in advance.

An **online search agent** is more appropriate because it:

- Explores the environment while moving.
- Collects new information using onboard sensors.
- Updates its knowledge after every action.
- Chooses the next action based on the latest observations.
- Re-plans whenever unexpected obstacles are detected.

II. Analyze the challenges of partial observability and dynamic environments

1. Partial Observability

The rover has limited sensing capability, meaning it can observe only a small area around its current position.

Challenges include:

- Hidden obstacles may exist beyond sensor range.
- The rover cannot accurately predict the safest path.
- It must estimate unknown regions using previously gathered information.
- Incorrect assumptions may lead to inefficient or dangerous routes.

Example:

A large crater may remain undetected until the rover approaches it, forcing it to change direction.

2. Dynamic Environment

Although Mars changes slowly, certain conditions can change unexpectedly.

Examples include:

- Dust storms reducing visibility.
- Loose soil causing wheel slippage.
- Small rocks shifting due to wind.
- Sensor noise causing inaccurate readings.

Challenges include:

- Previously safe paths may become unsafe.
- The rover must frequently update its route.
- Decisions must be made quickly to prevent accidents.
- Continuous replanning increases computational complexity.

III. Describe how the rover builds and updates its internal model

internal world model which stores information collected during exploration.

Step 1: Initial Knowledge

Initially, the rover knows:

- Mission objectives.
- Current position.

Step 2: Sense the Environment

Using cameras, lidar, radar, and other sensors, the rover observes:

- Rocks
- Craters
- Terrain elevation

Step 3: Update the Internal Map

After every observation, the rover:

- Marks explored regions.
- Records obstacle locations.

Step 4: Plan the Next Action

Based on the updated model, the rover selects the safest and most efficient next move.

Step 5: Repeat

IV. Compare online search with uninformed and informed search strategies

feature	online search	uninformed search	informed search
environment knowledge	unknown	usually known	usually known
uses heuristic	optional	no	yes
decision time	during execution	before execution	before execution
adaptability	very high	very low	moderate
handles unknown terrain	yes	no	limited
memory requirement	moderate	high	high
re-planning ability	excellent	poor	moderate
suitable for mars rover	yes	no	partially

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

The **Online Search Agent** is the most suitable approach for the Mars rover because it can effectively handle uncertainty, adapt to changing conditions, and prioritize safe navigation.

Justification

1. Handles Uncertainty

- Operates without requiring a complete map.
- Learns about the environment through exploration.
- Makes decisions based on current observations.

2. Highly Adaptable

- Re-plans routes whenever new hazards are detected.
- Adjusts to changing terrain and environmental conditions.
- Continuously improves navigation as more information becomes available.

3. Ensures Safety

- Detects obstacles before moving into dangerous areas.
- Avoids craters, rocks, and unstable terrain.
- Selects safer alternative routes when necessary.

4Q)

Scenario: AI-Based University Exam Timetable Generation Using Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

a constraint satisfaction problem (csp) consists of three main components:

- variables
- domains
- constraints

the objective is to assign a valid value to every variable while satisfying all constraints.

1. variables

variables represent the exams that need to be scheduled.

let:

- e_1 = mathematics exam
- e_2 = physics exam
- e_3 = chemistry exam
- e_4 = computer science exam
- e_5 = english exam
- ...
- e_n = nth course exam

ii. Explain how backtracking search works in this scenario

The domain contains all possible values that each variable can take.

For every exam:

- Available Time Slots
 1. Monday Morning
 2. Monday Afternoon
 3. Tuesday Morning
 4. Tuesday Afternoon

5. etc.

- Available Exam Halls

1. Hall A

2. Hall B

3. Hall C

4. Hall D

- Available Faculty Members

1. Faculty 1

2. Faculty 2

3. Faculty 3

Domain = {Time Slot $\times$ Hall $\times$ Faculty}

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

The constraints determine whether a timetable is valid.

Constraint 1: No Student Overlap

Constraint 2: Limited Exam Halls

Constraint 3: Subject Ordering

Constraint 4: Faculty Availability

Constraint 5: Hall Capacity

Constraint 6: Special Accommodation

Constraint 7: University Rules

iv. Analyze real-world challenges and justify the best strategy for scalability

Large universities face many practical challenges that increase the complexity of timetable generation.

1. Large Number of Students

Thousands of students may be enrolled in multiple courses, creating many potential exam conflicts.

2. Large Number of Courses

Hundreds of courses require scheduling across limited time slots.

3. Limited Infrastructure

Exam halls have fixed capacities, and only a limited number are available.

4. Faculty Constraints

Faculty members may have:

- Leave requests
- Administrative duties
- Teaching commitments

5Q)

Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

i. Explain how the Minimax algorithm models this problem

the minimax algorithm is a decision-making algorithm used in artificial intelligence for two-player competitive games. it assumes that both players are intelligent and always choose the

best possible move. the ai attempts to maximize its chances of winning, while the opponent tries to minimize the ai's advantage.

how minimax models the strategy game

- the current game situation is represented as the root node.
- every possible move that the ai can make forms the next level of the game tree.
- each response from the human player creates another level of the tree.
- the process continues until a predefined search depth or terminal state is reached.
- the ai evaluates the final states using an evaluation function.

- the values are then propagated upward:
 - max nodes (ai): choose the move with the highest evaluation value.
 - min nodes (opponent): choose the move with the lowest evaluation value.
- finally, the ai selects the move that guarantees the best possible outcome assuming the opponent always plays optimally.

ii. Analyze the computational challenges of large game trees

Real-time strategy games have extremely large search spaces because every unit may have multiple possible actions.

Major Computational Challenges

1. Exponential Growth

Total nodes: $O(b^d)$

This is computationally infeasible for real-time play.

2. Real-Time Constraints

Players expect responses within milliseconds.

The AI cannot spend several seconds evaluating millions of game states.

3. High Branching Factor

Unlike Chess:

- Many units move simultaneously.
- Every unit has multiple actions.
- Resource management adds more possibilities.

The number of legal actions becomes extremely large.

4. Memory Consumption

Every game state stores:

- Unit positions
- Health values
- Resources
- Buildings
- Technologies
- Terrain information

Millions of stored nodes require substantial memory.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

Alpha-Beta pruning is an optimization technique for Minimax.

It removes branches that cannot influence the final decision without changing the optimal result.

Key Idea

If a branch is guaranteed to be worse than an already explored alternative, there is no need to evaluate it.

Two Parameters

Alpha (α)

- Best value found so far for the maximizing player.
- Lower bound of possible outcome.

Beta (β)

- Best value found so far for the minimizing player.
- Upper bound of possible outcome.

Further exploration of that branch stops.

The remaining nodes are **pruned** because they cannot improve the final decision.

Benefits

- Reduces unnecessary computation.
- Returns exactly the same optimal move as Minimax.
- Enables deeper search within the same time limit.
- Makes real-time decision-making practical.

Complexity

Without pruning: $O(b^d)$

With ideal Alpha-Beta pruning: $O(b^{d/2})$

This significantly reduces the number of nodes evaluated.

iv. Design an evaluation function and justify the factors considered

Since the AI cannot search until the end of the game, it estimates the quality of intermediate positions using an evaluation function.

Factors Considered

1. Resource Advantage

Measures:

- Gold
- Wood
- Food
- Energy

More resources allow stronger armies and faster expansion.

2. Army Strength

Considers:

- Number of units
- Unit health
- Attack power
- Defense capability

A stronger army generally increases the chance of victory.

3. Map Control

Rewards:

- Capturing strategic locations
- Holding key objectives
- Controlling resource-rich areas

Better map control limits the opponent's options.

v. Recommend a strategy for balancing optimality and real-time performance

To perform effectively under strict time limits, the AI should combine several techniques instead of relying solely on full Minimax search.

Recommended Strategy

1. Use Minimax with Alpha-Beta Pruning to eliminate branches that cannot affect the final decision, reducing computation while preserving optimal results.
2. Apply Depth-Limited Search so the AI searches only a fixed number of levels and then evaluates positions using the evaluation function.

3. Use Iterative Deepening, starting with shallow searches and progressively increasing depth while always keeping the best move found so far. This ensures a valid move is available even if time expires.
4. Prioritize Move Ordering by exploring promising moves (such as attacks, captures, or strategic expansions) first, which increases the effectiveness of Alpha-Beta pruning.